

基于MQTT over TLS的矿井通信安全系统

项目名称： 矿井传感器安全通信系统

开发语言： Python 3.11+

协议标准： MQTT 3.1.1 + TLS 1.2

1. 目的

理解MQTT协议与TLS加密通信的原理与实现细节： 深入掌握 MQTT 协议规范和 TLS 双向认证机制。

掌握网络编程技术： 熟练使用 Python `paho-mqtt` 库进行 MQTT 客户端开发，实现传感器节点与地面中心的双向通信。

实现理论与实践结合： 将密码学理论（AES-GCM、数字证书、预共享密钥）应用于实际系统，培养网络安全问题分析与解决问题的能力。

构建完整系统： 实现传感器数据采集、加密传输、安全验证、本地启动编排与 Web 监控展示的完整链路。

2. 运行环境

2.1 硬件要求

指标	配置
CPU	Intel Core i5/i7, 4核及以上
内存	8GB及以上
网络带宽	100Mbps及以上

2.2 软件要求

软件	版本	用途
操作系统	Windows 10/11 / Ubuntu 20.04 / macOS	开发与运行环境
Python	3.11 - 3.13	核心开发语言
Mosquitto	2.0+	MQTT Broker
paho-mqtt	2.0+	MQTT客户端库
cryptography	42.0+	加密库
OpenSSL	1.1.1+	证书管理

2.3 依赖安装

```
python3 -m pip install -e '.[test]'
```

2.4 网络环境

局域网环境：传感器节点与地面中心通过局域网通信

TLS双向认证：客户端与服务端相互验证证书

端口配置：MQTT Broker 监听 8883 端口

3. 研究问题

3.1 核心问题

如何在矿井环境下实现安全可靠的传感器数据通信？

3.2 具体挑战

数据加密：如何在应用层实现 AES-GCM 加密，与 TLS 形成双重安全保障？

身份认证：如何使用预共享密钥 (PSK) 和证书实现传感器节点身份校验？

防重放攻击：如何防止攻击者重放历史消息？

断线检测：如何检测传感器节点意外断开并及时告警？

实时监控：如何实现 Web 端实时监控传感器数据？

3.3 研究目标

实现基于 MQTT over TLS 的安全通信系统

实现 AES-128-GCM 认证加密

实现序列号+时间戳的防重放机制

实现遗嘱消息 (LWT) 机制

实现 Web 实时监控界面

4. 系统设计与实现

4.1 系统架构

该系统为基于 MQTT over TLS 的矿井通信安全系统，采用三层架构实现井下数据的安全采集、传输与监控：

底层（传感器节点层）：部署于采煤工作面、掘进面等场景的传感器节点，负责温度、瓦斯浓度采集，通过 AES-GCM 加密数据后，以 MQTT 客户端身份向中间层发送消息，并支持 LWT 遗嘱消息保障离线状态感知；

中间层（MQTT Broker层）：由 Mosquitto Broker 承载，开放 8883 端口、运行 MQTT 协议，通过 **TLS 1.2 双向认证** 建立安全通道，作为“枢纽”转发底层与顶层的加密数据；

顶层（地面中心服务器）：依次完成“数据接收→解密→验证→告警”流程，并由本地启动器提供 Web 监控模块呈现实时状态、日志和告警。

4.2 核心代码实现

4.2.1 应用层加密模块

加密工具类 ([crypto_utils.py](#))：

```
def derive_aes128_key(psk_hex: str, sensor_id: str) -> bytes:
    """使用HKDF-SHA256从PSK派生每个传感器独立的AES-128密钥"""
```

```

psk = bytes.fromhex(psk_hex)
hkdf = HKDF(
    algorithm=hashes.SHA256(),
    length=KEY_SIZE,
    salt=sensor_id.encode('utf-8'),
    info=HKDF_INFO,
)
return hkdf.derive(psk)

def encrypt_payload(
    *,
    psk_hex: str,
    sensor_id: str,
    sensor_type: str,
    seq: int,
    timestamp_ms: int,
    plaintext: dict[str, Any],
    boot_random: bytes,
    version: int = 1,
) -> EncryptedPayload:
    """使用AES-128-GCM加密传感器读数"""
    key = derive_aes128_key(psk_hex, sensor_id)
    nonce = make_nonce(boot_random, seq)
    aad = build_aad(version, sensor_id, sensor_type, seq, timestamp_ms)
    encrypted = AESGCM(key).encrypt(nonce, canonical_json(plaintext), aad)
    ciphertext = encrypted[:-GCM_TAG_SIZE]
    tag = encrypted[-GCM_TAG_SIZE:]
    return EncryptedPayload(
        version=version,
        sensor_id=sensor_id,
        sensor_type=sensor_type,
        seq=seq,
        timestamp_ms=timestamp_ms,
        nonce=b64url_encode(nonce),
        ciphertext=b64url_encode(ciphertext),
        tag=b64url_encode(tag),
    )

```

加密机制特点:

- 使用 HKDF-SHA256 从主 PSK 派生每个传感器的独立密钥
- AES-GCM 提供机密性和完整性保护
- AAD（附加认证数据）包含版本号、传感器 ID、类型、序列号和时间戳
- Nonce 由 64 位启动随机数 + 32 位序列号组成，确保唯一性

4.2.2 TLS双向认证

MQTT客户端TLS配置 ([mqtt_runtime.py](#)) :

```

def make_tls_client(
    *,
    client_id: str,
    ca_file: str,
    cert_file: str,
    key_file: str,

```

```

) -> mqtt.Client:
    """创建已配置mTLS的paho-mqtt客户端"""
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id=client_id)
    client.tls_set(
        ca_certs=ca_file,
        certfile=cert_file,
        keyfile=key_file,
        tls_version=ssl.PROTOCOL_TLS_CLIENT,
    )
    client.tls_insecure_set(False)
    return client

```

Mosquitto Broker配置 (mosquitto.conf) :

```

listener 8883
protocol mqtt

cafile certs/ca.crt
certfile certs/broker.crt
keyfile certs/broker.key

require_certificate true
use_identity_as_username true
allow_anonymous false
tls_version tlsv1.2

```

4.2.3 防重放攻击机制

序列号验证 (replay_guard.py) :

```

class ReplayGuard:
    """校验时间戳新鲜度和重复序列号"""

    def __init__(self, window_ms: int = 300_000) -> None:
        self.window_ms = window_ms # 5分钟时间窗口
        self._states: dict[str, SensorReplayState] = {}

    def check(self, sensor_id: str, seq: int, timestamp_ms: int, receive_time_ms: int) -> str | None:
        # 时间戳新鲜度检查
        if abs(receive_time_ms - timestamp_ms) > self.window_ms:
            return 'timestamp_out_of_window'

        state = self._states.setdefault(sensor_id, SensorReplayState())
        self._prune(state, receive_time_ms)

        # 序列号重复检查
        if seq in state.recent_seq:
            return 'replay_detected'

        # 序列号回退检查
        if seq <= state.last_seq:
            return 'sequence_rollback'

        return None

```

防重放策略：

时间窗口验证：消息时间戳与接收时间差不超过5分钟

序列号重复检测：维护最近接收的序列号集合

序列号回退检测：拒绝序列号小于已接收最大序列号的消息

4.2.4 地面中心处理核心

消息处理逻辑 ([center_core.py](#))：

```
class GroundCenterCore:
    """供MQTT回调和测试复用的纯校验核心"""

    def __init__(
        self,
        *,
        psk_map: dict[str, str],
        sensor_types: dict[str, str],
        thresholds: dict[str, dict[str, float]],
        replay_guard: ReplayGuard | None = None,
    ) -> None:
        self.psk_map = psk_map
        self.sensor_types = sensor_types
        self.thresholds = thresholds
        self.replay_guard = replay_guard or ReplayGuard()

    def process_data_message(
        self,
        payload: dict[str, Any],
        *,
        certificate_identity: str | None,
        receive_time_ms: int,
    ) -> ProcessResult:
        sensor_id = str(payload.get('sensor_id', ''))
        sensor_type = str(payload.get('sensor_type', ''))

        # 身份验证
        if sensor_id not in self.psk_map or sensor_id not in self.sensor_types:
            return self._reject('unknown_sensor', sensor_id, {'sensor_id': sensor_id})

        # 证书身份校验（当前默认运行链路通常拿不到对端证书CN，因此这里只在集成层显式提供时生效）
        if certificate_identity is not None and certificate_identity != sensor_id:
            return self._reject('identity_mismatch', sensor_id, {
                'certificate_identity': certificate_identity,
                'payload_sensor_id': sensor_id,
            })

        # 传感器类型校验
        if self.sensor_types[sensor_id] != sensor_type:
            return self._reject('identity_mismatch', sensor_id, {
                'expected_sensor_type': self.sensor_types[sensor_id],
                'payload_sensor_type': sensor_type,
            })

        # 重放检测
```

```

replay_error = self.replay_guard.check(sensor_id, seq, timestamp_ms, receive_time_ms)
if replay_error:
    return self._reject(replay_error, sensor_id, {'seq': seq, 'timestamp_ms': timestamp_ms})

# AES-GCM解密
plaintext = decrypt_payload(self.psk_map[sensor_id], payload)

# 阈值告警检测
value = plaintext.get('value')
if isinstance(value, int | float):
    alert = threshold_alert(sensor_id, sensor_type, float(value), self.thresholds)
    if alert is not None:
        alerts.append(alert)

return ProcessResult(accepted=True, plaintext=plaintext, alerts=alerts)

```

处理流程:

传感器身份验证 (PSK映射检查)
 证书身份校验 (可选; 默认运行链路通常由Broker负责接入层校验)
 传感器类型校验
 重放攻击检测 (序列号+时间戳)
 AES-GCM解密与完整性验证
 阈值告警检测

4.2.5 遗嘱消息(LWT)机制

传感器节点配置 ([sensor_cli.py](#)) :

```

offline_payload = encode_json({
    'sensor_id': args.sensor_id,
    'status': 'offline',
    'reason': 'unexpected_disconnect',
})
client.will_set(status_topic(args.sensor_id), offline_payload, qos=1, retain=True)
client.connect(host, port, keepalive=60)

# 发布在线状态
client.publish(status_topic(args.sensor_id), encode_json({
    'sensor_id': args.sensor_id,
    'status': 'online',
    'timestamp_ms': now_ms(),
}), qos=1, retain=True)

```

地面中心处理:

```

def process_status_message(self, payload: dict[str, Any]) -> ProcessResult:
    sensor_id = str(payload.get('sensor_id', ''))
    status = payload.get('status')
    if status == 'offline':
        return ProcessResult(
            accepted=True,
            alerts=[Alert(

```

```

        code='sensor_offline',
        sensor_id=sensor_id,
        severity='high',
        message='sensor unexpectedly disconnected',
        details=payload,
    )],
)
return ProcessResult(accepted=True, plaintext=payload)

```

4.2.6 瓦斯超限告警

阈值配置 ([sensors.toml](#)) :

```

[thresholds.gas]
warning = 1.0
critical = 1.5

[thresholds.temperature]
warning = 45.0
critical = 60.0

```

告警检测逻辑 ([alerts.py](#)) :

```

def threshold_alert(
    sensor_id: str,
    sensor_type: str,
    value: float,
    thresholds: dict[str, dict[str, float]],
) -> Alert | None:
    sensor_threshold = thresholds.get(sensor_type, {})
    critical = sensor_threshold.get('critical')
    warning = sensor_threshold.get('warning')

    if critical is not None and value >= critical:
        return Alert(
            code=f"{sensor_type}_threshold_exceeded",
            sensor_id=sensor_id,
            severity='high',
            message=f"{sensor_type} critical threshold exceeded",
            details={'value': value, 'threshold': critical},
        )

    if warning is not None and value >= warning:
        return Alert(
            code=f"{sensor_type}_threshold_warning",
            sensor_id=sensor_id,
            severity='medium',
            message=f"{sensor_type} warning threshold exceeded",
            details={'value': value, 'threshold': warning},
        )

    return None

```

4.2.7 Web实时监控

监控页面 ([web/index.html](#)) :

- 系统状态展示 (Broker、中心、传感器状态)
- 温度/瓦斯实时数据显示
- 传感器列表与状态管理
- 实时日志与告警展示
- 系统测试功能 (基于当前状态快照的前端检查)

API接口 ([start_system.py](#)) :

```
def do_GET(self) -> None:
    parsed = urlparse(self.path)
    if parsed.path == '/api/status':
        self._send_json(self.launcher_state.snapshot())
        return
    if parsed.path == '/api/sensors':
        self._send_json(self.launcher_state.frontend_sensor_map())
        return
    self._serve_static(parsed.path)
```

4.3 安全机制总结

安全层	机制	实现方式
传输层	TLS 1.2双向认证	客户端/服务端证书验证
应用层	AES-128-GCM加密	认证加密 (AEAD)
密钥管理	HKDF-SHA256密钥派生	每个传感器独立密钥
身份认证	PSK + mTLS接入控制	PSK映射 + Broker证书接入校验
防重放	序列号+时间戳	5分钟时间窗口
完整性	GCM认证标签	AAD保护元数据
断线检测	LWT遗嘱消息	MQTT KeepAlive机制

5. 项目文件结构

```
mine_sensor_secure_comm/
├─ src/mine_sensor_secure_comm/      # 核心模块
│   ├── __init__.py                 # 包入口
│   ├── crypto_utils.py             # AES-GCM加密工具
│   ├── message.py                  # MQTT消息辅助函数
│   ├── mqtt_runtime.py             # MQTT客户端配置
│   ├── sensor_node.py              # 传感器节点类
│   ├── sensor_sim.py               # 传感器模拟器
│   ├── sensor_cli.py               # 传感器命令行入口
│   ├── center_core.py              # 地面中心核心逻辑
│   ├── center.py                   # 地面中心入口
│   ├── alerts.py                   # 告警模型
│   └─ replay_guard.py               # 防重放防护
```


		config_loader.py	# 配置加载工具
		log_records.py	# 日志记录器
		benchmark.py	# 性能基准测试
		security_tests.py	# 安全场景测试
		scripts/	# 启动脚本
		start_system.py	# 一键启动器
		start_mosquitto.bat/sh	# Mosquitto启动脚本
		generate_certs.bat/sh	# 证书生成脚本
		config/	# 配置文件
		sensors.toml	# 传感器配置
		psk.json.example	# PSK示例配置
		psk.json	# 运行时PSK配置
		mosquitto.conf	# Mosquitto配置
		certs/	# TLS证书
		ca.crt/key	# CA根证书
		broker.crt/key	# Broker证书
		center.crt/key	# 地面中心证书
		sensor.crt/key	# 传感器证书
		web/	# Web监控界面
		index.html	# 监控页面
		app.js	# 前端逻辑
		style.css	# 样式文件
		tests/	# 单元测试
		test_crypto_utils.py	# 加密模块测试
		test_replay_guard.py	# 防重放测试
		test_center_core.py	# 中心核心测试
		test_start_system.py	# 启动器测试
		doc/	# 文档
		system_design.md	# 系统设计文档
		deployment.md	# 部署说明
		pyproject.toml	# Python项目配置
		README.md	# 项目说明

6. 功能实现总结

6.1 已实现功能

功能	说明	状态
多传感器节点模拟	支持温度、瓦斯传感器	✓
TLS 1.2双向认证	客户端与服务端相互验证	✓
AES-128-GCM加密	认证加密（机密性+完整性）	✓
防重放攻击机制	序列号+时间戳双重验证	✓
遗嘱消息(LWT)	传感器意外断开检测	✓
瓦斯超限告警	基于阈值的告警系统	✓
Web实时监控	实时数据展示与日志	✓
启动器日志持久化	JSONL运行日志与会话目录管理	✓

6.2 安全特性

特性	描述
✔ 双向认证	TLS客户端证书验证
✔ 数据加密	AES-128-GCM应用层加密
✔ 完整性保护	GCM认证标签
✔ 防重放	序列号+时间窗口
✔ 断线检测	LWT遗嘱消息
✔ 密钥派生	HKDF-SHA256

7. 参考资料

7.1 RFC/标准文档

- RFC 5246 - The Transport Layer Security (TLS) Protocol Version 1.2
- MQTT 3.1.1 Specification - OASIS Standard
- NIST SP 800-38D - Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC
- NIST SP 800-56C - Recommendation for Key Derivation Using HMAC-Based Extract-and-Expand Key Derivation Function (HKDF)

7.2 开源文档

- Eclipse Paho MQTT Client Documentation
- cryptography Python Library Documentation
- Mosquitto Broker Documentation

附录：配置示例

PSK配置 (psk.json)

```
{
  "gas_sensor_01": {
    "psk_id": "gas_sensor_01",
    "psk_hex": "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
  },
  "temperature_sensor_01": {
    "psk_id": "temperature_sensor_01",
    "psk_hex": "11112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
  }
}
```

传感器配置 (sensors.toml)

```
[simulation]
```

```
default_interval_seconds = 0.5
locations = ["采煤面", "掘进面", "回风巷", "运输巷", "水泵房"]

[units]
gas = "%CH4"
temperature = "°C"

[sensors.gas_sensor_01]
type = "gas"
unit = "%CH4"
location = "采煤面"
interval_seconds = 0.5
client_cert = "certs/gas_sensor_01.crt"
client_key = "certs/gas_sensor_01.key"

[thresholds.gas]
warning = 1.0
critical = 1.5

[mqtt]
host = "localhost"
port = 8883
ca_file = "certs/ca.crt"
center_cert = "certs/center.crt"
center_key = "certs/center.key"
```